

Why do we need State Management?

When we build a web application, communication between **Client (Browser)** and **Server** happens using **HTTP protocol**.

But **HTTP is a *stateless* protocol**.

✅ **Stateless means:**

- The **server does NOT remember** who you are.
- Every request looks like it is coming from a **new user**, even if it is the same user.

So if the user logs in and then goes to another page, the server **forgets** the user login data.

Therefore...

To **remember** the user across multiple requests, we use **State Management**.

State management helps the server **identify the user**, **store data**, and **remember** it while the user is using the website.

State Management Types

Type	Where Data is Stored?	Used For	Example in Flask
Client-Side	Browser	Store small data at user's side	Cookies, LocalStorage
Server-Side	Server	Store user session details securely	Sessions

1) Client-Side State Management (Cookies)

- Cookies are stored in the **browser**
- Data is visible to user
- Can be modified by user
- Used for: Remember username, language, small preferences

Flask Example:

```
from flask import Flask, make_response
```

```
app = Flask(__name__)
```

```
@app.route('/set_cookie')  
  
def set_cookie():  
    res = make_response("Cookie Set")  
    res.set_cookie("username", "Jani")  
    return res
```

```
@app.route('/get_cookie')  
  
def get_cookie():  
    from flask import request  
    user = request.cookies.get("username")  
    return f"Cookie Value = {user}"
```

2) Server-Side State Management (Sessions)

- Data is stored on **server**
- Safer and more secure
- Used for: login session, cart session, user data, etc.

Flask Example:

vs

Simple Explanation Conclusion

HTTP is Stateless → Server forgets user → So we use State Management.

Cookies → Data stored in Browser → Client-Side.

Sessions → Data stored in Server → Server-Side.

What is a Cookie?

A **cookie** is a **small piece of data** stored in the browser.

- Stored **on the client side** (browser)
- Used to remember **small and non-sensitive information**
- It is visible and can be modified by the user → **Less Secure**

Example Use Cases:

- Remembering **dark/light theme**
- Remembering **last visited page**
- Remembering **language (English / Telugu / Hindi)**

Flask Example (Cookie):

```
from flask import Flask, make_response, request
```

```
app = Flask(__name__)
```

```
@app.route('/set')
```

```
def set_cookie():
```

```
    res = make_response("Cookie Stored")
```

```
    res.set_cookie("name", "Jani")
```

```
    return res
```

```
@app.route('/get')
```

```
def get_cookie():
```

```
    name = request.cookies.get("name")
```

```
    return f"Hello {name}"
```

```
app.run(debug=True)
```

✓ Data stored inside browser.

● What is a Session?

A **session** stores data **on the server** (not in the browser).

- Used for **login and user authentication**
- More secure than cookies
- Browser stores **only session ID**, not real data

Example Use Cases:

- Remembering who is logged in
- Keeping items in shopping cart
- User profile information

Flask Example (Session):

```
from flask import Flask, session
```

```
app = Flask(__name__)
```

```
app.secret_key = "abc"
```

```
@app.route('/login')
```

```
def login():
```

```
    session['user'] = "Jani"
```

```
    return "Logged in"
```

```
@app.route('/dashboard')
```

```
def dashboard():
```

```
    return f"Welcome {session['user']}"
```

app.run(debug=True)

✓ Data stored inside **server memory**, not browser.

🔥 Main Difference (Very Important)

Feature	Cookie	Session
Where stored?	Browser (Client)	Server
Security	✗ Less secure	✓ More secure
Data size	Small (4 KB)	Large (depends on server)
Used for	Simple preferences Login / user data	
Can user modify?	Yes	No

Important:

✓ When user logs in:

Flask stores the **user's session data on the server**, and

Flask also **creates a cookie in the browser** named: **session**

That cookie contains **only the Session ID (encrypted)**, not the actual username or password.

So what happens:

Place	What is Stored	Example
Server	Actual session data (session['username'])	{'username': 'admin'}
Browser	Cookie named session containing encrypted Session ID	eJytVltv....

Why does Flask create this cookie?

Because:

- Every time the browser sends a new request (ex: going dashboard → logout → profile)
- Flask needs **a way to identify which user** is sending the request

So the browser sends the **session cookie** automatically with each request, and Flask uses its `secret_key` to **decode it** and find the correct user session.

Final Simple Sentence

When the user logs in, Flask stores session data on the server and creates a **cookie named session** in the browser that contains an **encrypted session ID** to identify the user in future requests.

Browser stores → Cookie named "session"

Inside → Encrypted Session ID

Now the question is:

When does this session cookie get removed from the browser?

There are 3 main cases.

CASE 1: Session Expiry

You set this in your code:

```
app.permanent_session_lifetime = timedelta(milliseconds=500)
```

Meaning:

If the user does not make any request within 500 ms (0.5 sec),
the session will expire.

When the session expires:

The session data on server is deleted

The session cookie in browser becomes useless (invalid)

 The cookie may still be visible in browser cookies list,
but it cannot be used anymore — it is expired.

 CASE 2: User Logs Out

In your logout route:

```
session.pop('username', None)
```

This removes the session data from the server.

But note:

The cookie may still stay in the browser for a short time

But it becomes worthless (because its session no longer exists)

So the user will be redirected to login again.

✅ CASE 3: User Manually Clears Cookies

If the user opens browser settings → Clear Cookies

→ The session cookie is completely removed.

This is manual removal.

🎯 Summary Table (Easy to Remember)

Case	What happens to Cookie?	Can user still access dashboard?
Session Expired	Cookie becomes invalid	❌ No
User Logs Out	Cookie may still remain but useless	❌ No
User Clears Browser Cookies	Cookie fully removed	❌ No

✅ Final 1-Line Answer for Interview

The session cookie (session) in the browser is removed or becomes invalid when the session expires, when the user logs out, or when the browser cookies are cleared.

Important :

🚦 Why Do We Need State Management?

When a user interacts with a website, multiple requests are sent from the **client (browser)** to the **server**.

But **HTTP is a stateless protocol**, meaning:

The server does not remember who made the previous request.

So the server treats every request as **new**.

Example:

1. User logs in → **First Request**
2. User clicks Dashboard → **Second Request**

Without state management → Server **forgets** the user → asks to login again → ❌ Bad experience.

So, To **remember the user**, we use **State Management**.

✅ What is State Management?

State Management is the process of remembering user data across multiple page requests.

🎯 Types of State Management

There are **2 major types**:

Type	Where data is stored?	Example Techniques
Client-Side State Management	On Browser	Cookies, LocalStorage, SessionStorage
Server-Side State Management	On Server	Sessions, Database

1) 🟡 Client-Side State Management

→ **Data is stored on the browser.**

a) Cookies

- Small text data stored in browser.
- Used for simple information like username or preferences.

Flask Example:

```
from flask import Flask, make_response, request
```

```
app = Flask(__name__)
```

```
@app.route('/set_cookie')

def set_cookie():

    resp = make_response("Cookie Set!")

    resp.set_cookie("username", "Jani")

    return resp
```

```
@app.route('/get_cookie')

def get_cookie():

    user = request.cookies.get("username")

    return f"Hello {user}"
```

```
app.run(debug=True)
```

✅ Data stored in browser → user can see and modify.

2) ● Server-Side State Management (Sessions)

→ **Data is stored on server memory / database, not on browser.**

Browser only stores a **session ID**, not real data.

Flask Example:

```
from flask import Flask, session, redirect, render_template
```

```
app = Flask(__name__)

app.secret_key = "abc123"
```

```
@app.route('/login')

def login():

    session['user'] = "Jani"

    return "Logged In"
```

```
@app.route('/dashboard')
def dashboard():
    if 'user' in session:
        return f"Welcome {session['user']}"
    return redirect('/login')
```

```
@app.route('/logout')
def logout():
    session.pop('user')
    return "Logged Out"
```

```
app.run(debug=True)
```

✅ More secure than cookies.

🏆 Comparison (Very Important)

Feature	Cookies	Session
Stored at	Browser	Server
Security	Less secure	More secure
Storage Limit	Small (4 KB)	Depends on server
Good For	Simple data (names, theme, language)	Login info, user-specific data

🎉 Simple Real-Life Example

Situation	Best Option	Why?
Remember website theme (dark/light)	Cookie	Small and non-sensitive
Remember logged-in user until logout	Session	Sensitive info, must be secure

Final Easy Answer (For Exam & Interview)

State Management is used to remember user data between multiple requests because HTTP is stateless.

It is of two types:

1. **Client-Side State Management** → Data stored in browser (Cookies)
2. **Server-Side State Management** → Data stored on server (Sessions)